CBAS-DSH

Software Simulation and Reproducibility Package

Comprehensive End-to-End Execution Guide

From reviewer-safe source release to build, deployment, benchmark, evidence, and cleanup

Build-validated repository state. The clean Maven workflow completes successfully, executes the automated tests, packages the application, and publishes `scheme.war` as a downloadable artifact. This statement concerns software build validation and does not claim formal cryptographic verification.

Contents

1 Purpose and verification scope

This guide provides a complete, reviewer-oriented procedure for inspecting and executing the source code accompanying the CBAS-DSH manuscript. It supports four independent checks:

1. verify the integrity and automated build of the supplied source;
2. run the JUnit correctness and tamper-rejection tests;
3. deploy the Spring MVC web simulation and call its REST endpoints; and
4. reproduce the descriptive statistics reported in Table 6.

The package compares the baseline CBAS construction attributed to Qiao et al. with the proposed CBAS-DSH construction under the same Spring MVC, JPBC, controller instrumentation, and client conditions.

2 Before sharing while the manuscript is under review

Determine the venue's review model before distributing the repository link.

Review model	**Required handling**
Single-anonymous	A named public repository may be used if the venue permits external artifacts. The reviewer already knows the author identity.
Double-anonymous	Use an anonymous repository or upload an anonymous ZIP through the manuscript system. Remove author names, affiliations, personal usernames, e-mail addresses, ORCID identifiers, funding acknowledgments, and identifying commit history.
Unknown	Ask the editor or consult the venue's author guidelines before sharing any identifying URL.

Do not include the submitted manuscript PDF, confidential editor correspondence, reviewer comments, rebuttal letters, manuscript number, credentials, or private database configuration in the public code package. The guide uses `REPOSITORY_URL` as a neutral placeholder so it can be used in either review model.

3 End-to-end verification map

The recommended order is fixed so that failures are isolated early:

1. check review-policy compliance and package structure;
2. install and verify Java, Maven, Python, and Tomcat;
3. verify the bundled JPBC checksums;
4. perform a clean Maven build and automated test run;
5. inspect the generated WAR and continuous-integration evidence;
6. deploy the WAR to Tomcat 9;
7. execute smoke tests against all four reported REST operations;
8. repeat the 100-run experiment and retain the new CSV;
9. independently analyze the supplied raw CSV to reproduce Table 6;
10. archive evidence, record deviations, and cleanly stop the environment.

4 Package contents

Location	Purpose
`src/main/java/com/abc/service/impl/VehicleServiceImpl.java`	Qiao and CBAS-DSH algorithm implementation
`src/main/java/com/abc/controller/VehicleController.java`	REST endpoints and controller-time instrumentation
`src/test/java/.../VehicleServiceImplTest.java`	Correctness, tamper, nonce, domain-separation, and PoP tests
`src/main/webapp/`	Browser benchmark interface and web configuration
`benchmark_cbas.py`	Automated runner and CSV statistics calculator
`data/raw/table6_100_runs.csv`	Raw observations used to reconstruct Table 6
`libs/`	Vendored JPBC 2.0.0 modules and SHA-256 checksums
`THIRD_PARTY_LICENSES/`	JPBC LGPL 3.0 license text
`.github/workflows/build.yml`	Automated clean build, test, and WAR publication

Generated directories such as `target/` and `__pycache__/` are not part of the reviewed source and should not be committed.

5 Prerequisites

Component	Required configuration
Java	JDK 17
Build tool	Apache Maven 3.9 or newer
Web container	Apache Tomcat 9 (Servlet 4 / `javax.servlet`)
Benchmark client	Python 3.8 or newer; no third-party Python packages required
Network	Access to Maven Central for standard dependencies

Tomcat 10 or newer uses the `jakarta.servlet` namespace and is not compatible with this evaluated Spring MVC 5 configuration without migration. Use Tomcat 9 for direct reproduction.

Confirm the local tools:

```
java -version
mvn -version
python3 --version
```

5.1 Install the prerequisites

On macOS with Homebrew:

```
brew install openjdk@17 maven python@3.12
brew install --cask temurin@17
```

Install Tomcat 9 from the official Apache Tomcat distribution and extract it to a path without unusual permission restrictions. On Ubuntu/Debian, a typical JDK/Maven/Python setup is:

```
sudo apt update
sudo apt install openjdk-17-jdk maven python3 python3-venv
```

On Windows, install a JDK 17 distribution, Apache Maven, Python 3, and Tomcat 9. Add the JDK `bin`, Maven `bin`, and Python directories to `PATH`. Set `JAVA_HOME` to the JDK 17 installation, not a JRE.

Verify which Java Maven actually uses:

```
mvn -version
```

The output must report Java 17. A different Java version in `mvn -version` indicates an environment configuration problem even when `java -version` appears correct.

6 Obtain the source

Clone the public repository and enter its root directory:

```
git clone REPOSITORY_URL
cd REPOSITORY_DIRECTORY
```

Alternatively, use GitHub's **Code** > **Download ZIP**, extract the archive, and open a terminal in the extracted repository root. The correct root contains `pom.xml`, `README.md`, `src/`, `libs/`, and `data/`.

Confirm the package is complete:

```
ls -la
find libs data src/test .github -maxdepth 3 -type f | sort
```

On Windows PowerShell, use `Get-ChildItem -Recurse`. Do not run Maven from the directory above or below the repository root.

7 Verify the bundled JPBC libraries

JPBC 2.0.0 is not reliably available from public Maven repositories. The package therefore vendors only the two modules imported by the implementation: `jpbc-api` and `jpbc-plaf`. The GitHub workflow checks them automatically before compiling.

On macOS or Linux, verify them manually from the repository root:

```
shasum -a 256 -c libs/SHA256SUMS
```

On Linux systems with GNU Coreutils, the equivalent command is:

```
sha256sum --check libs/SHA256SUMS
```

Expected result:

```
libs/jpbc-api-2.0.0.jar: OK
libs/jpbc-plaf-2.0.0.jar: OK
```

The applicable license is retained at `THIRD_PARTY_LICENSES/JPBC-LGPL-3.0.txt`. Maven copies both JARs into `WEB-INF/lib` in the generated WAR.

8 Clean build and automated tests

From the repository root, execute:

```
mvn --batch-mode --no-transfer-progress clean verify
```

A successful run ends with `BUILD SUCCESS` and produces:

```
target/scheme.war
```

The `verify` phase compiles the application, executes the JUnit suite, and creates the deployable WAR. The tests cover valid Qiao and CBAS-DSH signing/verification, aggregate operations, modified-message and modified-signature rejection, deterministic nonce recomputation, domain-separated hashing, and proof-of-possession validation.

Record the Maven summary, including the test count, failures, errors, skipped tests, total time, and final `BUILD SUCCESS` line. A generated WAR alone is not sufficient evidence if the tests were skipped.

8.1 Inspect the WAR without deploying it

A WAR is a ZIP-compatible Java web archive. Verify that it is readable and contains the application classes, web configuration, and both JPBC JARs:

```
unzip -t target/scheme.war
unzip -l target/scheme.war | grep -E \
  'WEB-INF/(web.xml|classes|lib/jpbc)'
```

On Windows PowerShell, inspect the archive with:

```
jar tf target/scheme.war | Select-String "WEB-INF"
```

Do not rename the file to `.zip` and do not deploy an extracted directory when validating the supplied packaging procedure.

Database note. The four reported cryptographic operations do not query the legacy DAO. A running MySQL server is therefore not required for the build, automated tests, REST benchmark, or Table 6 reconstruction.

9 Verify through GitHub Actions

Every push or pull request invokes `.github/workflows/build.yml`. A reviewer may open the repository's **Actions** tab and select the newest **Build and Test** run. The expected indicators are:

- green **Success** status;
- successful JPBC checksum verification;
- successful Maven clean verification; and
- one artifact named `scheme-war` containing `scheme.war`.

Download the workflow artifact when a local Maven installation is unavailable. GitHub packages downloadable artifacts as ZIP files; extract that download once to obtain `scheme.war`. Do not extract the WAR itself.

For reproducible reporting, record the commit identifier, workflow run number, workflow status, execution date, Java version, and artifact name. Earlier failed workflow runs may remain visible in history; the evidence relevant to the evaluated source is the successful run associated with the exact commit supplied to reviewers.

10 Deploy the web simulation on Tomcat 9

10.1 Manual deployment

1. Download and extract Apache Tomcat 9.
2. Copy `target/scheme.war`, or the WAR downloaded from GitHub Actions, into Tomcat's `webapps/` directory.
3. Start Tomcat using the command for the host operating system.

macOS or Linux:

```
cp target/scheme.war /path/to/apache-tomcat-9/webapps/
/path/to/apache-tomcat-9/bin/startup.sh
```

Windows Command Prompt:

```
copy target\scheme.war C:\path\to\apache-tomcat-9\webapps\
C:\path\to\apache-tomcat-9\bin\startup.bat
```

Wait until Tomcat expands the application, then open:

```
http://localhost:8080/scheme/
```

For a foreground run with immediately visible logs on macOS/Linux, use:

```
/path/to/apache-tomcat-9/bin/catalina.sh run
```

Keep this terminal open during endpoint and benchmark testing. Use a separate terminal for `curl` and Python commands.

Stop Tomcat with `shutdown.sh` on macOS/Linux or `shutdown.bat` on Windows.

10.2 Confirm deployment

The benchmark page should display controls for the Qiao and CBAS-DSH sign/verify operations. If the page does not load, inspect `logs/catalina.out` on macOS/Linux or the current Tomcat console on Windows.

11 Call the REST endpoints

The four operations used for Table 6 are:

Scheme	Operation	Method	Endpoint
Qiao	Sign	POST	`/scheme/vehicle/sign-qiao`
Qiao	Verify	POST	`/scheme/vehicle/verify-qiao`
CBAS-DSH	Sign	POST	`/scheme/vehicle/sign-dsh`
CBAS-DSH	Verify	POST	`/scheme/vehicle/verify-dsh`

Example request:

```
curl -X POST http://localhost:8080/scheme/vehicle/verify-dsh \
  -H "Content-Type: application/json" \
  -d '{"name":"vehicle-01"}'
```

A valid JSON response includes a success code, a true/accepted result, and `controllerTimeMs`. This backend metric, rather than browser/network elapsed time, is the measurement reported in

Table 6.

11.1 Four-operation smoke test

Call each reported operation once before starting the 100-run experiment:

```
for op in sign-qiao verify-qiao sign-dsh verify-dsh; do
  curl --fail-with-body -sS -X POST \
    "http://localhost:8080/scheme/vehicle/$op" \
    -H "Content-Type: application/json" \
    -d '{"name":"vehicle-01"}'
  echo
done
```

Each request must return HTTP success and a valid application result. Do not begin the measured run if any endpoint returns an error, invalid result, HTML error page, or connection refusal.

12 Repeat the 100-run experiment

Keep Tomcat running and execute:

```
python3 benchmark_cbas.py \
  --base-url http://localhost:8080/scheme/vehicle \
  --runs 100 \
  --output results.csv
```

The runner invokes the four operations sequentially, writes one CSV row per request, and prints the sample count, mean, median, minimum, maximum, and sample standard deviation. The manuscript protocol uses 100 measured requests per operation with no separate unrecorded warm-up.

Verify the new result file immediately:

```
python3 benchmark_cbas.py --analyze results.csv
wc -l results.csv
```

The CSV should contain one header plus 400 measurement rows when all four operations complete successfully. If any row records a request or validity failure, preserve the file for diagnosis but do not present that run as a successful reproduction.

Results on different hosts may vary because the measured controller path includes JPBC Type A parameter generation, key and certificate setup, signing, verification, JVM behavior, and garbage collection. Preserve the generated CSV and report any changed hardware, JDK, or warm-up configuration.

13 Reproduce Table 6 without deploying Tomcat

The supplied raw observations can be analyzed independently:

```
python3 benchmark_cbas.py --analyze data/raw/table6_100_runs.csv
```

Expected summary:

Operation	n	Mean	Median	Min	Max	SD
Sign-Qiao	100	92.24	84.50	66	216	27.74
Verify-Qiao	100	113.08	104.00	90	183	21.56
Sign-DSH	100	100.94	95.00	81	165	18.00
Verify-DSH	100	126.25	119.00	105	275	24.61

All times are backend controller execution times in milliseconds. The corresponding mean overhead is 9.43% for Sign (8.70 ms) and 11.65% for Verify (13.17 ms).

14 Evidence package and reporting template

Retain the following evidence for a complete reviewer record:

- the exact repository commit identifier or immutable source ZIP;
- output of `java -version`, `mvn -version`, and `python3 -version`;
- JPBC checksum verification output;
- complete Maven test/build log;
- `scheme.war` or its workflow artifact reference;
- Tomcat version and deployment log;
- responses from the four smoke-test endpoints;
- the newly generated `results.csv`; and
- the analyzed output for both the new CSV and supplied Table 6 CSV.

Use the following compact report when communicating a reproduction result:

```
Source commit/archive :
Operating system      :
Hardware              :
JDK / Maven / Python  :
Tomcat                :
JPBC checksum         : PASS / FAIL
Maven tests           : PASS / FAIL
Deployment            : PASS / FAIL
Four endpoint smoke   : PASS / FAIL
100-run experiment    : PASS / FAIL
Table 6 reconstruction: PASS / FAIL
Deviations/notes      :
```

15 Stop, clean, and repeat

After testing, stop Tomcat cleanly. For a completely fresh repetition:

1. stop Tomcat;
2. preserve required logs and CSV evidence outside generated directories;
3. remove the old deployed `scheme.war` and expanded `webapps/scheme/`;
4. run `mvn clean`;
5. rebuild with `mvn clean verify`; and
6. redeploy only the newly generated WAR.

Never delete the supplied `data/raw/table6_100_runs.csv`, `libs/`, checksum manifest, license, source files, or the new benchmark CSV intended as evidence.

16 Troubleshooting

Symptom	Resolution
`Could not find artifact jpbc-*`	Confirm that `libs/jpbc-api-2.0.0.jar` and `libs/jpbc-plaf-2.0.0.jar` exist, then use the supplied `pom.xml`. Do not replace them with unresolved remote JPBC dependencies.
Checksum failure	Re-download or clone the repository. Do not use a modified JAR.
Java class-version error	Run both Maven and Tomcat with JDK 17. Check `java -version` and `mvn -version`.
HTTP 404 for `/scheme/`	Confirm the WAR is named `scheme.war`, is inside Tomcat's `webapps/`, and Tomcat has completed deployment.
Servlet/Jakarta class errors	Use Tomcat 9, not Tomcat 10+.
Port 8080 already in use	Stop the conflicting service or change Tomcat's Connector port in `conf/server.xml`; update the benchmark base URL accordingly.
Exact times differ	Expected across machines. Verify the operation ordering, 100-run count, zero warm-up, JDK, and controller metric.

For a clean retry, stop Tomcat, remove the expanded `webapps/scheme/` directory and old `scheme.war`, then deploy the newly built WAR.

17 Reviewer completion checklist

☐ The repository root and key source locations were inspected.
☐ Both JPBC JAR checksums returned `OK`.
☐ `mvn clean verify` ended with `BUILD SUCCESS`.
☐ The automated test suite completed without failures.
☐ `target/scheme.war` or the GitHub Actions artifact was obtained.
☐ The application opened at `http://localhost:8080/scheme/`.
☐ The four individual REST endpoints returned valid JSON responses.
☐ A new 100-run CSV was retained, if the live experiment was repeated.
☐ Analysis of the supplied raw CSV reproduced Table 6 exactly.
☐ The commit/archive identifier and complete environment were recorded.
☐ Generated logs and the new 100-run CSV were retained as evidence.
☐ No author-identifying metadata was exposed when double-anonymous review applied.